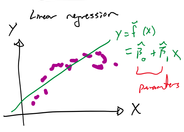
# Wk7.1 Decision trees

Tuesday, October 20, 2026 9:19 AM

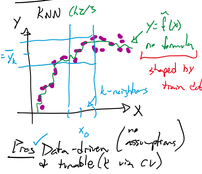
2 broad categories of methods in SL: Regression $Y = f(X) + \varepsilon$ ← Q8

**Parametric** vs **Non-parametric**

Linear regression: $Y = \hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X$ (parameters)



kNN (k=3): $\hat{y}_0 = \bar{y}_{N_0}$, $\hat{f}(x)$ not formula, shaped by train data, k-neighbors



Pros: Nice $\hat{f}$ to write, what is interpretable & simple

Cons: Linear assumption may be "wrong" (too simple) — High bias

Pros: Data-driven (no assumptions) & tunable (k via CV)

Cons: Hard to interpret, sensitive to train data — High variance

Add decision trees to non-parametric methods. Improve on knn (interpretable)

## Decision trees Regression case

Hitters salary → $Y = f(X) + \varepsilon$, $X = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$




High level DT idea:

① All data in one bucket

② Split into 2 buckets in best way possible (product of min RSS) — All possible $X_j + X_k$ splits




③ Predict $\hat{y}_{R_j} = \bar{y}_{R_j}$

④ Iterate ② split more until stop condition is met

(i) Min leaf node size (other rules)

(ii) Split completely (each leaf 1 sample) & prune up via CV.

Interpretable

① Root split is most significant (Yrs makes most)

② Subtree gives subset among (old vs young)

## Detailed Version

$Y = f(X) + \varepsilon$, $X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_p \end{bmatrix}$

All $X$ train data is in region $R_{All}$

① Consider all possible partitions of $R_{All}$

$$R_1(j, s) = \{X \mid X_j < s\}$$ ← predictor to split ← threshold

$$R_2(j, s) = \{X \mid X_j > s\}$$

Loop over all $X_j$'s & all threshold cuts $s$ (= all midpoints)

② Per partition possibility, compute predictions $\hat{y}$ for

$$\hat{y}_{R_1} = \bar{y}_{R_1} \quad \& \quad \hat{y}_{R_2} = \bar{y}_{R_2}$$

indicator: 0, False; 1, true

$$\& \quad \hat{y} = \hat{f}(X) = \sum_{i=1}^{2} \hat{y}_{R_i} \mathbb{I}(X \in R_i) = \sum_{i=1}^{2} \bar{y}_{R_i} \mathbb{I}(X \in R_i)$$

③ Compute RSS per partition.

$$RSS(j, s) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

④ Choose partition & which minimizes RSS

⑤ Repeat for 2 new nodes

⑥ Iterate until stop condition.

Once a model is trained, how to predict & interpret?

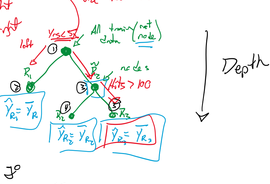
**Predict**

New player: $X_0 = \begin{bmatrix} x_{10} \\ x_{20} \end{bmatrix} = \begin{bmatrix} 10 \\ 50 \end{bmatrix}$, $\hat{y}_0 = ?$

① yrs = 10 > 5, move right

② Hits = 50 < 100, move right

Predict $\hat{y}_0 = \hat{y}_{R_3} = \bar{y}_{R_3}$




**Interpret**

Top split most important, less as go down. Quantify by RSS gains.

## Balance overfitting & underfitting

① CV ✓

② Regularization

CAS: $\hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \ldots + \hat{\beta}_p x_p$

Ridge: RSS + Shrinkage $= \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^{p} \hat{\beta}_i^2$

LASSO: " " $= " " + \lambda \sum_{i=1}^{p} |\hat{\beta}_i|$

penalize big $\hat{\beta}_i$'s

↳ Balance RSS & shrinkage (# of leaves)

$$\sum_{i=1}^{n} (y_i + \hat{y}_i)^2 + \text{shrinkage} \quad \text{shrinkage} \propto \text{\# of leaves}$$

$$= \sum_{m=1}^{|T|} \sum_{y_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

# of leaves in $T$

$\alpha$: prune parameter to tune via CV.

## Note (i)

Partitioning $X$ via DTs is not restrictive for regression (& classification).

Linear regress often is more accurate.

Instead use **ensembles**: (1000s of trees, subset samples & $X_j$'s, train data & avg predictions) to improve accuracy & boost parameterize models.

Random Forest: Bagging + Boosting + Gradient Boosting + Xgboost (CAS) — Easy to parallelize & randomly

## Decision tree for classification

Almost the same as DT for regression, but need a new way to decide splits.

Predict: majority class in region.

Split: to improve accuracy of predictions.

Entropy, poor class imbalance & multiclass.

CAS ① Gini coef TSC

② Entropy